

TEORÍA (8 puntos)**Apellidos y nombre:**

Pregunta 1. Las dos instrucciones siguientes, ¿son equivalentes? ¿es preferible una sobre la otra? Justifica brevemente tu respuesta.

```
assertTrue(x == 8);
```

```
assertEquals(8, x);
```

Pregunta 2. Técnicas basadas en puntos de decisión.

Dado el siguiente código fuente escrito en un lenguaje de operadores lógicos no cortocircuitados:

```
if a > 0 or b > 0
    then r = 0;
    else r = -1;
end if;
```

y los siguientes casos de prueba posibles:

Caso	a	b	Resultado esperado
1	5	5	r = 0
2	5	-2	r = 0
3	-2	5	r = 0
4	-2	-2	r = -1

Selecciona un conjunto mínimo de casos de prueba para conseguir un 100 % de cobertura de los siguientes tipos:

- Cobertura de decisiones:
- Cobertura de condiciones:
- Cobertura de condiciones y decisiones:
- Cobertura múltiple de condiciones:
- Cobertura de condiciones y decisiones modificada:

Pregunta 3. ¿Cuáles son las tres fases principales en el desarrollo guiado por las pruebas (*test driven development*, TDD)? ¿Qué color se suele asociar a dos de ellas?

Pregunta 4. Da tres motivos por los cuales se justifica la automatización de pruebas sobre la Interfaz de Usuario

Pregunta 5. Da tres ejemplos de riesgos del proceso de pruebas:

Pregunta 6. Asumiendo que estás realizando el análisis de riesgos de un producto, ¿qué factores debemos tener en cuenta para asignar una clase de riesgo A, B o C a un componente de una característica de calidad cuando asignamos la clase de riesgo a los distintos objetivos de prueba asociados con esa característica de calidad?

Pregunta 7. ¿Qué es una organización permanente de pruebas y qué ventajas proporciona dentro de una empresa de desarrollo de *software*?

Pregunta 8. Describe brevemente la técnica de estimación basada en la Extrapolación

Pregunta 9. Describe el tipo de cobertura y la información requerida en la base de las pruebas para aplicar la técnica de prueba de pares.

Pregunta 10. Explica brevemente cómo se puede estimar el rendimiento o beneficio de las pruebas desde un punto de vista económico.

EJERCICIO (2 puntos)**Prueba de caminos con profundidad**

Diseña pruebas para el método `list()` utilizando la técnica de pruebas de caminos con **nivel de profundidad 2**. El código es parte de la implementación de una clase denominada `Hotel` que tiene un atributo denominado `rooms` que es una lista de listas de enteros y representa las habitaciones disponibles en el hotel, organizadas por plantas. Cada sublista corresponde a una planta específica, donde el índice de la lista externa representa el número de la planta, y la lista interna contiene los números de las habitaciones disponibles en esa planta.

Un objeto de la clase `StringBuilder` permite manejar cadenas de caracteres de forma eficiente. Su constructor genera un objeto que representa la cadena vacía. Se puede agregar texto con el método `append()` sin que se generen múltiples objetos intermedios. El método `isEmpty()` evalúa si un `StringBuilder` está vacío (es decir, no contiene todavía ningún texto).

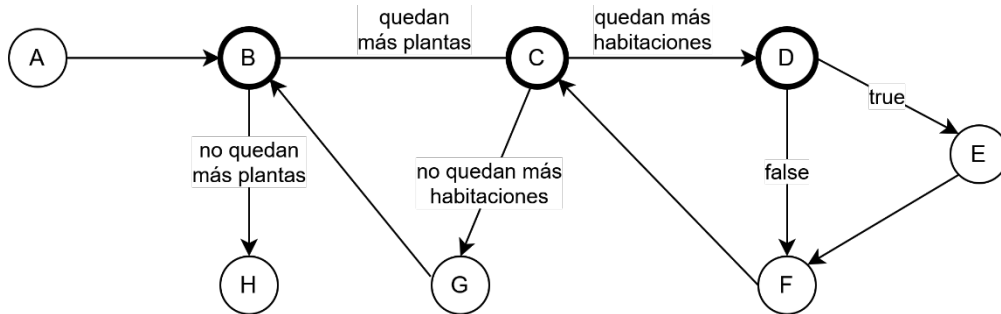
1. Obtén primero el grafo de flujo asociado al método `list()`, de acuerdo con la correspondencia entre instrucciones/expresiones y nodos que se indica en el propio código (0,3 puntos).
Los bucles *for each* (correspondientes a los nodos B y C), pueden interpretarse como «mientras quedan plantas en el atributo `rooms`...» y «mientras quedan habitaciones en la planta `floor`...», respectivamente.
2. Hay dos aristas de ese grafo en las que está involucrado el nodo predicado D que presentan dependencias entre sí, aunque todas las situaciones de prueba que generan son perfectamente posibles. Indica de qué aristas se trata y en qué consiste la dependencia entre ellas (0,3 puntos).
3. Aplica la técnica de pruebas de caminos con profundidad 2 para obtener el conjunto de caminos completos correspondiente (0,7 puntos). Al hacerlo, ten en cuenta la dependencia entre las dos aristas mencionadas anteriormente.
4. Finalmente, establece casos de prueba para los caminos obtenidos, incluyendo entradas y salidas del método (0,7 puntos).

```
public class Hotel {  
    private final List<List<Integer>> rooms;  
  
    public String list() {  
        AStringBuilder result = new StringBuilder();  
        for ( B List<Integer> floor : rooms ) {  
            for ( C int room : floor ) {  
                if ( D !result.isEmpty() ) {  
                    E result.append(",");  
                }  
                F result.append(room);  
            }  
            G }  
            H return result.toString();  
        }  
        ...  
    }  
}
```

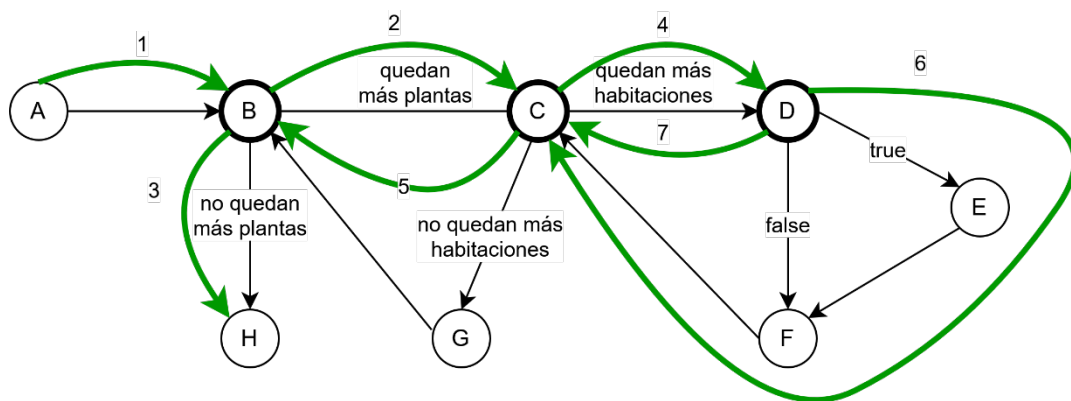
Solución

Grafo

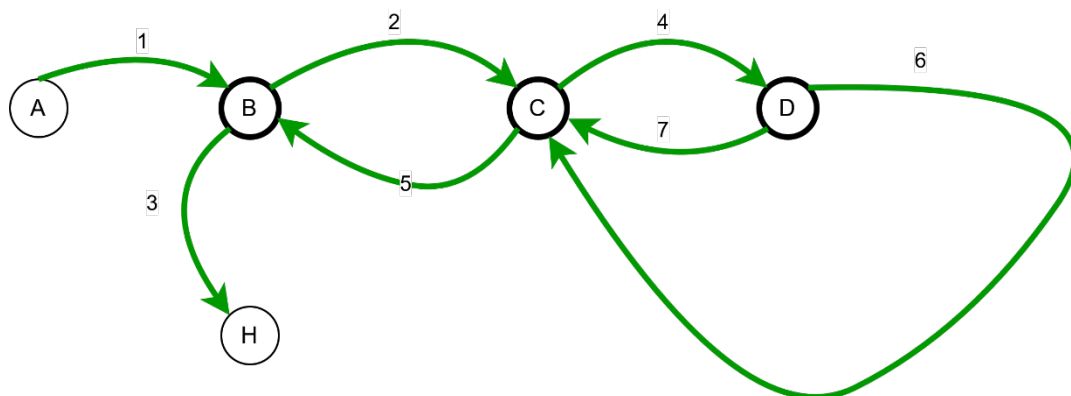
Se muestra a continuación el grafo que representa el flujo del programa. Las aristas no se han numerado todavía y los nodos predicado se han destacado para darles visibilidad:



A continuación, se añaden al grafo arcos que van de nodo predicado o inicial a nodo predicado o final:



Por legibilidad, se muestra ahora el grafo exclusivamente con las aristas que van de nodo predicado a nodo predicado. Este grafo puede ayudar a la hora de identificar las situaciones de prueba, pero al haber eliminado nodos, complica el identificar los casos de prueba concretos una vez establecidos los caminos. En una respuesta de examen, bastaría con incluir el grafo anterior.



Dependencias entre aristas

La única vez en cada ejecución del método `list()` en la que el nodo predicado D (`!result.isEmpty()`) se evalúa como falso es la primera vez que se ejecuta el cuerpo del bucle más interno, que es cuando `result` representa la cadena vacía y, por lo tanto, `result.isEmpty()` devuelve `true`, ejecutándose el camino DFC (arista 7). En el resto de las ejecuciones del cuerpo de este bucle, `!result.isEmpty()` se va a evaluar como cierto y se va a ejecutar el camino DEFC (arista

6). Por lo tanto, en todos los casos de prueba en los que se alcance el nodo D, se ejecutará primero la arista 7 una sola vez. Después, se ejecutará solo la arista 6 en cero, una o más ocasiones.

Situaciones de prueba

Profundidad 2 (12 situaciones)

- 12
- 13
- 24
- 25
- 46
- 47
- 52
- 53
- 64
- 65
- 74
- 75

Caminos y casos de prueba

Caminos prof. 2		Condiciones	rooms	Resultado
C1	124746465253	2 plantas: 3 habs., 0 habs.	{{4, 5, 6}, {}}	"4,5,6"
C2	13	0 plantas	{}	""
C3	124753	1 planta: 1 habitación	{{4}}	"4"

Criterios generales de corrección

- No hacer bien el grafo de flujo por no considerar exclusivamente como aristas las que van de nodo predicado a nodo predicado: 0 puntos por el grafo y 50 % en la aplicación de la técnica (con un máximo de 0,3 puntos).
- Aplicación de la técnica: 0,25 la identificación correcta de situaciones de prueba (0,02 por situación de prueba correctamente identificada si no hay situaciones de más y el grafo estaba correctamente hecho y numerado; 0,25 si están todas) y 0,45 generar caminos de acuerdo con las situaciones de prueba (que todas estén incluidas al menos una vez, que no se repitan si no es innecesario, ...) (0,15 puntos por camino correctamente identificado).
- Casos de prueba: En general: 0,25 puntos por camino si se plantean 3 caminos o menos (máx. 0,7); 0,7/n si se plantean (o debieran haberse planteado, en función de las situaciones de prueba) $n > 3$ caminos. Para obtener los puntos, el caso de prueba tiene que hacer que el flujo en el programa sea el marcado por el camino correspondiente y los resultados esperados deben reflejarse correctamente.